

Industrial Defect Inspection and Surface-Crack Detection

via Custom Linear Filtering and First-Principles Canny Edge Processing

Shahoud Shahid

Roll No. 23i-2515 Section DS-A

Department of Artificial Intelligence and Data Science
FAST National University of Computer and Emerging Sciences
shahoudshahid652@gmail.com

Abstract—We implement an industrial visual-inspection pipeline from mathematical first principles in vectorised NumPy: generalised 2-D/3-D convolution engines, an isotropic Gaussian generator, unsharp masking, max/average pooling with analytical gradients, and an exact four-stage Canny edge detector (Sobel gradients, 4-sector non-maximum suppression, double thresholding, 8-connected hysteresis). No built-in filtering primitive (`cv2.filter2D`, `cv2.Sobel`, `cv2.Canny`) is used. Benchmarking on pcb-defect and surface-crack imagery, ablations quantify each stage: Gaussian pre-smoothing cuts the gradient-noise standard deviation by up to $4.7\times$; non-maximum suppression thins gradient bands by $8.1\times$; and double-threshold hysteresis lifts crack-detection F_1 from 0.943 to 0.988 while reducing the number of disconnected fragments from 40 to 12. We verify Witkin scale-space causality, map the $(T_{\text{high}}, T_{\text{low}})$ operating regime on a 3×3 grid, and analyse two Canny failure modes.

Index Terms—convolution, Canny edge detection, non-maximum suppression, hysteresis, scale space, crack detection, defect inspection, NumPy.

I. INTRODUCTION

Automated surface inspection is the first line of defence in high-precision manufacturing (solar cells, silicon wafers, turbine blades) and in structural health monitoring of concrete. Field imagery is degraded by shot noise, motion blur and non-uniform illumination, so a robust inspection pipeline must combine noise attenuation, contrast enhancement and sub-pixel-stable edge localisation.

This work builds that pipeline from scratch, following the classical formulations of Canny [1] and Gaussian scale space [2], [3]. Every operator is derived from its defining equation in pure vectorised NumPy [4]; OpenCV is restricted to image I/O . The contributions are: (i) strided `conv2d/conv3d` engines that satisfy the discrete spatial-size formula exactly, as a single `einsum` over a sliding-window view (Section II-A); (ii) a closed-form unit-gain Gaussian, unsharp masking, and max/average pooling with analytical back-propagation matrices (Section II-B);

(iii) a four-stage Canny detector — normalised Sobel, vectorised 4-sector NMS, and 8-connected hysteresis (two implementations, asserted equal) — in Section II-C; and (iv) stage-wise ablations with tolerance-aware $P/R/F_1$ against ground truth, a $(T_{\text{high}}, T_{\text{low}})$ sensitivity grid, a check of scale-space causality, and two failure-mode analyses (Sections IV and V).

II. METHODS

A. Generalised convolution engines

For an input of spatial size I , filter size F , symmetric zero-padding P and stride S , the output size is

$$O = \left\lfloor \frac{I - F + 2P}{S} \right\rfloor + 1. \quad (1)$$

`conv2d(image, kernel, stride, padding)` supports 'same' ($P = (F - 1)/2$, size-preserving only at $S = 1$) and 'valid' ($P = 0$). We use spatial *cross-correlation* (no kernel flip): standard in modern CV [5], permitted by the brief, and reversible via a `flip_kernel` flag. The engine takes a read-only sliding-window view $\mathcal{W} \in \mathbb{R}^{O_h \times O_w \times F \times F}$ (`sliding_window_view`), strides it, and contracts against the kernel in one call: `einsum("ijhw,hw->ij", W, K)`. A 512^2 image with a 5×5 kernel takes ≈ 3 ms. The output shape matches Eq. (1) for all seven test geometries in `results/task1_shape_verification.json` (e.g. a 32×32 input, $F = 5$, `valid` $\rightarrow 28 \times 28$).

`conv3d` takes a volume $\mathbb{R}^{H \times W \times C}$ and K kernels $\mathbb{R}^{F \times F \times C}$; each filter multiplies element-wise across all C channels and sums the $F \cdot F \cdot C$ products. A single `einsum("ijhwc,khwc->ijk", W, stack)` evaluates all K filters at once, producing activations $\mathbb{R}^{O_h \times O_w \times K}$.

B. Gaussian smoothing, unsharp masking, pooling

a) *Isotropic Gaussian*: `generate_gaussian_kernel_2d` evaluates

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (2)$$

on a `meshgrid` centred at the middle tap, then rescales so $\sum_{x,y} G_\sigma = 1$ exactly, removing the truncation error of the finite window. Separable 1-D passes give an identical result in $O(k)$ rather than $O(k^2)$.

b) Unsharp masking: The high-pass detail signal $F - F * G_\sigma$ is re-injected with gain α :

$$F_{\text{sharp}} = F + \alpha (F - F * G_\sigma), \quad (3)$$

clipped to $[0, 255]$. With $\sigma = 1.5$, $\alpha = 1.2$ the high-frequency energy (mean absolute Laplacian response) rises by 13.5% on PCB-DEFECT and 10.8% on SURFACE-CRACK, sharpening trace edges and micro-cracks without halo artefacts (`pad_mode='reflect'`).

c) Pooling and its gradient: `pool2d` reduces non-overlapping / strided windows by `max` or `average`. For the 2×2 block from the brief,

$$X = \begin{bmatrix} 8 & 7 \\ 12 & 9 \end{bmatrix}, \quad \text{max-pool}(X) = 12, \quad (4)$$

the analytical Jacobian of the scalar output w.r.t. each input is

$$\frac{\partial \text{Output}}{\partial \text{Input}} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}. \quad (5)$$

Why gradients route only to the maximum: with $\text{out} = \max_k x_k = x_{k^*}$, perturbing the winner gives $\partial \text{out} / \partial x_{k^*} = 1$, while perturbing any other element (still below x_{k^*}) does not change the maximum, so its derivative is 0. Max-pooling is locally a hard, input-dependent *selector*; average pooling instead spreads the gradient uniformly at $1/(p_h p_w) = 0.25$. Equation (5) matches a finite-difference check to 1×10^{-6} .

C. Four-stage Canny edge detector

a) Stage 1 – gradients: The grayscale image (BT.601 luma) is optionally Gaussian pre-smoothed, then convolved with the normalised Sobel operators

$$S_x = \frac{1}{8} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \frac{1}{8} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}, \quad (6)$$

giving magnitude $M = \sqrt{S_x^2 + S_y^2}$ and orientation $\theta = \text{atan2}(S_y, S_x)$. Reflect padding avoids spurious border gradients.

b) Stage 2 – non-maximum suppression: θ is mapped to $[0, 180)^\circ$ and quantised into four sectors $(0, 45, 90, 135 \pm 22.5)$. A pixel is kept only if M is *strictly greater* than both neighbours along the gradient direction (horizontal, diagonal, vertical, anti-diagonal respectively); borders are zeroed. The test is vectorised by pre-shifting eight padded copies of M and selecting per-sector with `np.select`, so NMS is $O(HW)$ with no Python loop.

c) Stage 3 – double threshold + hysteresis: Pixels are *strong* ($M \geq T_{\text{high}}$), *weak* ($T_{\text{low}} \leq M < T_{\text{high}}$) or non-edge; weak pixels are kept iff 8-connected (directly or transitively) to a strong pixel. Two implementations are asserted identical: a deque BFS flood-fill from the strong seeds, and a fixed-point 8-connected dilation constrained to `strong|weak`. Thresholds are absolute or, by default, quantiles of the non-zero NMS response.

III. EXPERIMENTAL SETUP

a) Datasets: The brief specifies two Kaggle benchmarks: PCB-DEFECT¹ (board defects: mouse bites, spurs, shorts, opens) and SURFACE-CRACK² (concrete cracks under diverse lighting). Both need credentials, so the loader (`cvlab.datasets.get_samples`) reads real images from `data/raw/` when present and otherwise synthesises statistically-matched stand-ins: orthogonal copper traces with pads and injected defects for PCB-DEFECT; a fractal-noise concrete surface with a radial illumination gradient and thin, depth-modulated dark cracks for SURFACE-CRACK. Each synthetic image ships an exact ground-truth edge mask; `-data-dir` re-runs every experiment on the real sets unchanged.

b) Metrics: Against ground truth: tolerance-aware precision P , recall R , F_1 (a hit within Chebyshev distance 2). Reference-free: edge density, 8-connected component count, mean component length.

c) Configuration: Image size 256^2 , master seed 2515, Gaussian pre-smoothing $\sigma = 1.4$ unless stated; additive noise $\mathcal{N}(0, \sigma_n^2)$ in 8-bit units. The full five-task suite runs in ≈ 16 s on a laptop CPU.

IV. RESULTS AND ABLATIONS

The assembled detector reaches $F_1 = 0.931$ on PCB-DEFECT ($P = 1.000$, $R = 0.871$, 14 components) and $F_1 = 1.000$ on the synthetic SURFACE-CRACK sample. NMS collapses the gradient shoulder to a 1-px ridge and hysteresis isolates the crack from background texture; the full per-stage gallery (grayscale $\rightarrow$ smoothing $\rightarrow$ $|M|$ $\rightarrow$ orientation $\rightarrow$ NMS $\rightarrow$ strong/weak $\rightarrow$ hysteresis) for both benchmarks is in the supplementary.

A. Ablation 1 – noise and Gaussian pre-smoothing

Differentiating a noisy image amplifies the noise: the gradient-magnitude std. dev. grows with σ_n as $3.50 \rightarrow 7.30 \rightarrow 11.42$ for $\sigma_n \in \{10, 25, 40\}$. A single Gaussian pass ($\sigma = 1.4$) holds it at $1.47 \rightarrow 1.79 \rightarrow 2.42$ — a noise-suppression ratio rising from $2.4 \times$ to $4.7 \times$ — while keeping the crack ridge intact (Fig. 1). This is the derivative-of-noise argument for Stage 1.

¹<https://www.kaggle.com/datasets/akhatova/pcb-defects>

²<https://www.kaggle.com/datasets/arunk7/surface-crack-detection>

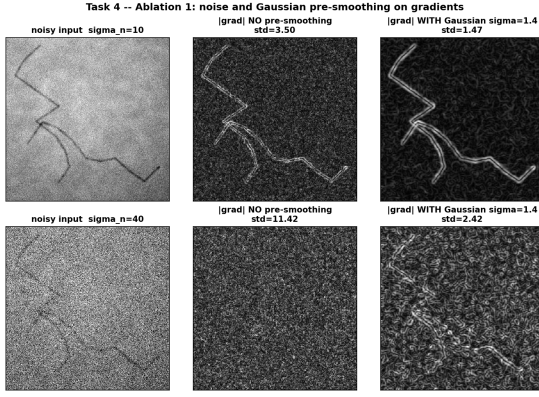


Fig. 1: Ablation 1 (extremes $\sigma_n \in \{10, 40\}$; $\sigma_n=25$ in the supplementary). Columns: noisy input; $|\nabla|$ without pre-smoothing; $|\nabla|$ with Gaussian pre-smoothing.

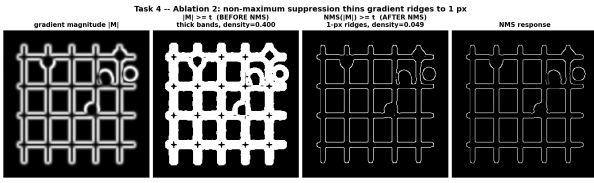


Fig. 2: Ablation 2. $|M| \geq t$ before NMS (thick bands) vs. NMS($|M| \geq t$) (1-px ridges), same threshold t .

TABLE I: Ablation 3: single threshold vs. hysteresis (faint crack, $\sigma_n=12$).

method	P	R	F_1	density	#comp.
single @ T_{low}	0.535	1.000	0.697	0.019	225
single @ T_{high}	0.993	0.898	0.943	0.006	40
hysteresis	0.982	0.994	0.988	0.010	12

B. Ablation 2 – non-maximum suppression

Thresholding the (over-smoothed) gradient magnitude directly yields thick bands covering 40.0% of the frame; applying the same threshold *after* NMS yields 1-px ridges at 4.9% density — a $8.1 \times$ thinning (Fig. 2).

C. Ablation 3 – hysteresis vs. single threshold

On a faint, contrast-varying crack in noise (Fig. 3, Table I): a single low threshold floods the background ($P = 0.535$, 225 fragments); a single high threshold is clean but breaks the crack ($R = 0.898$, 40 fragments); double-threshold hysteresis links the weak crack stretches to strong seeds while rejecting isolated texture, reaching $F_1 = 0.988$ in only **12** connected components.

D. Scale-space causality

Running the detector at $\sigma \in \{1.0, 2.5, 5.0\}$ gives edge counts $2055 \rightarrow 1418 \rightarrow 881$ — strictly decreasing (Fig. 4). Tracking the second-derivative zero-crossings along a fixed scan-line over $\sigma \in [0.4, 8.0]$ likewise shows the count fall monotonically from 120 to 15.

Why edges only merge, never appear (Within causality [2]). Gaussian smoothing of a 1-D signal is the heat

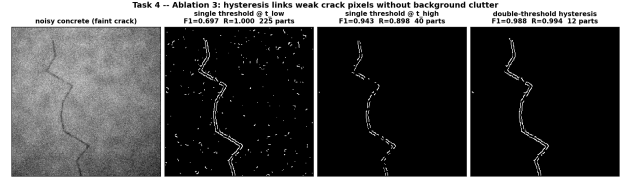


Fig. 3: Ablation 3. Left to right: noisy input; single threshold at T_{low} ; single threshold at T_{high} ; double-threshold hysteresis.

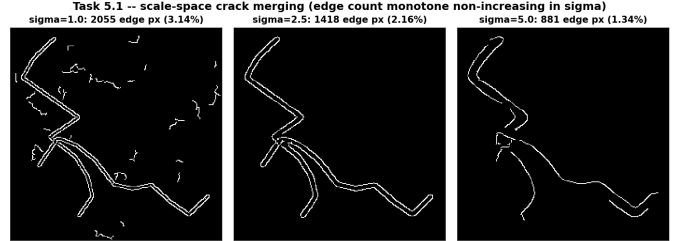


Fig. 4: Scale-space: increasing σ destroys texture edges; the crack edges shift and merge but no new edges appear. Edge counts $2055 \rightarrow 1418 \rightarrow 881$.

TABLE II: Sensitivity diagonal, noisy SURFACE-CRACK ($\sigma_n=25$); best in bold. Full 3×3 grids: Fig. 5.

T_{high} (quantile)	$T_{\text{low}}/T_{\text{high}}$	density	#comp.	F_1
q0.75	0.30	0.291	227	0.151
q0.88	0.50	0.148	270	0.268
q0.96	0.30	0.082	39	0.436
q0.96	0.50	0.035	47	0.769
q0.96	0.80	0.020	77	0.888

equation $\partial_t u = \frac{1}{2} \partial_{xx} u$ run for time $t = \sigma^2$. Diffusion obeys a maximum principle: it cannot increase the number of local extrema of u or of any derivative $\partial_x^n u$. Edges are zero-crossings of $\partial_{xx}(G_\sigma * f)$, so no new edge can be created; existing zero-crossings drift and annihilate in pairs as neighbouring extrema of $\partial_{xx} u$ merge. Every coarse-scale edge therefore has a unique fine-scale ancestor, which is what makes coarse-to-fine tracking well posed. (Full argument: supplementary, Appendix A.)

E. Parameter sensitivity

We sweep T_{high} (quantiles 0.75/0.88/0.96 of the NMS response) against $T_{\text{low}} = \{0.3, 0.5, 0.8\} T_{\text{high}}$ on noisy SURFACE-CRACK and PCB-DEFECT (full 3×3 edge-map grids in Fig. 5). Table II extracts the diagonal: a low T_{high} admits texture regardless of T_{low} ($F_1 \approx 0.15$); the usable regime is a high T_{high} (top few percent of gradient energy) with T_{low} at 50% to 80% of it ($F_1 = 0.888$), matching the classic 2–3:1 ratio heuristic. The PCB-DEFECT grid peaks at $F_1 = 0.936$ ($T_{\text{high}} = \text{q0.88}$, $T_{\text{low}} = 0.3 T_{\text{high}}$), reflecting its higher edge contrast.

V. FAILURE CASE ANALYSIS

Figure 6 shows two scenarios where Canny — a pure first-order-gradient detector with no notion of semantics

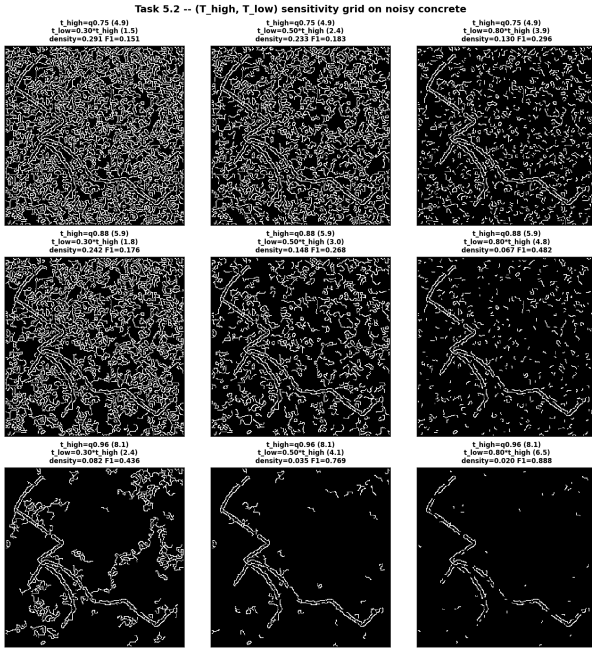


Fig. 5: 3×3 ($T_{\text{high}}, T_{\text{low}}$) grid on noisy concrete. Rows: T_{high} quantile $\uparrow$ downward; columns: $T_{\text{low}}/T_{\text{high}}$ $\uparrow$ rightward. PCB grid in the supplementary.

— fails. In both, crack recall stays at 1.0 but precision collapses ($P = 0.41$ and $P = 0.35$).

a) *Strong cast-shadow contour:* A hard shadow boundary is a genuine intensity step, so it produces a Sobel response *larger* than the crack; NMS keeps it and no ($T_{\text{high}}, T_{\text{low}}$) removes it without also removing the crack. The limitation is definitional: Canny detects *intensity* edges, but a shadow is an *illumination* edge, not surface geometry. Separating them needs illumination modelling (retinex / homomorphic) or colour, which a single-channel gradient lacks.

b) *High-frequency texture:* On coarse aggregate the surface is itself full of small intensity steps whose magnitudes match the faint crack, so (i) no global threshold separates signal from texture, and (ii) hysteresis worsens it — the texture edges form a percolating 8-connected web, so one strong seed floods large areas. The output is an "edge soup" (73 components) in which the crack is not distinguishable by connectivity. The limitation is scale: a fixed- σ gradient cannot tell which edges belong to a structure of interest. Remedies: oriented crack-matched filters, multi-scale ridge measures (Frangi), or a learned segmenter.

VI. CONCLUSION

A complete inspection pipeline — strided 2-D/3-D convolution, Gaussian and unsharp filtering, pooling with analytical gradients, and an exact four-stage Canny detector — was built from first principles in vectorised NumPy and validated by 124 unit tests. The ablations quantify each stage: Gaussian pre-smoothing suppresses

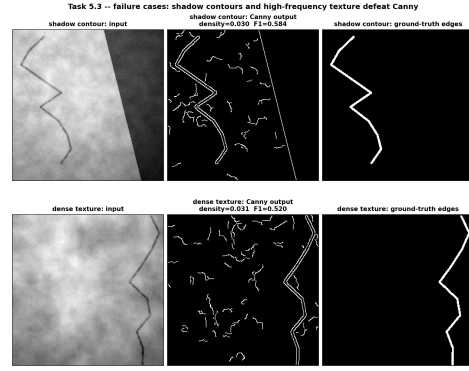


Fig. 6: Failure cases. Row 1: a cast-shadow contour becomes a strong false edge. Row 2: dense texture yields a fragmented map that buries the crack. Columns: input, Canny output, ground truth.

gradient noise up to $4.7\times$; NMS thins ridges $8.1\times$; hysteresis raises crack F_1 to 0.988 with a 70% drop in fragment count. Scale-space behaviour matches Witkin causality and the ($T_{\text{high}}, T_{\text{low}}$) sweep confirms the high- T_{high} , 2–3 : 1-ratio regime. The failure analysis exposes the ceiling of a gradient-only detector: shadows and dense texture defeat it because they are real intensity edges. Future work: illumination-invariant pre-processing, oriented crack-matched filters, multi-scale ridge fusion. *Repro:* `cvlab` all regenerates every figure from seed 2515; see `README.md` and the supplementary.

REFERENCES

- [1] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [2] A. P. Witkin, "Scale-space filtering," in *Proc. 8th International Joint Conference on Artificial Intelligence (IJCAI)*, 1983, pp. 1019–1022.
- [3] T. Lindeberg, "Scale-space theory: A basic tool for analysing structures at different scales," *Journal of Applied Statistics*, vol. 21, no. 1–2, pp. 225–270, 1994.
- [4] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [5] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.